Потоки в сетях — девять практических задач

Дискретная математика · Тоцкий В., гр. 5151003/40001

Задача 1. Существование потока меньшей величины

Условие. *если в сети G существует поток величины x, то для любого y с условием $0 \le y < x$ существует поток величины y.*

Что пишем на доске и рисуем.

По теореме о разложении поток — это сумма потоков по путям s → t и потоков по циклам; величина потока складывается только из путевых компонент, а циклы не влияют. Отсюда идея: путевую часть потока f можно масштабировать на любой коэффициент от 0 до 1 и получить поток меньшей величины, не нарушая пропускные способности.

Пусть f — поток величины x. По теореме о разложении:

$$f = \alpha_1 \cdot P_1 + \alpha_2 \cdot P_2 + \ldots + \alpha_k \cdot P_k + \beta_1 \cdot C_1 + \ldots + \beta_m \cdot C_m, \quad \Sigma\, \alpha_i = x.$$

Положим

$$f' = (y / x) \cdot (\alpha_1 \cdot P_1 + \ldots + \alpha_k \cdot P_k).$$

Проверка:

— линейная комбинация путевых потоков — поток;

— $|f'| = (y/x) \cdot \Sigma\, \alpha_i = y$;

— $y/x < 1 \implies f'(e) \le f(e) \le c(e)$ для всех рёбер, ограничения выполнены. ■

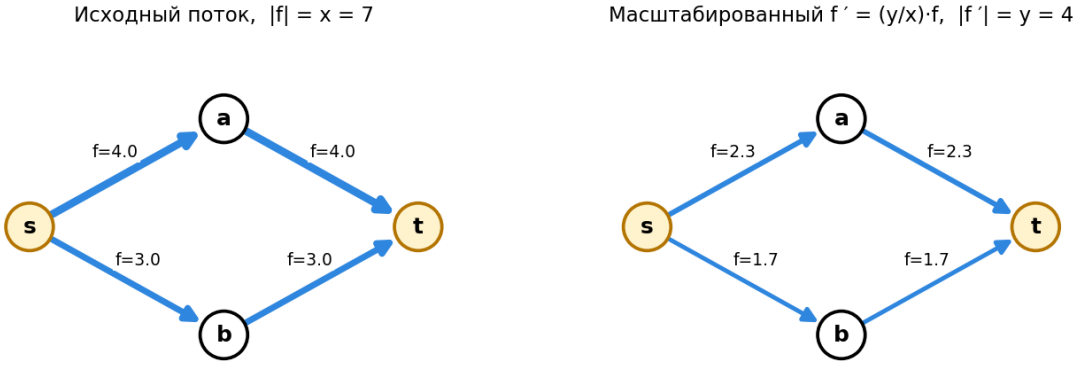

Рисунок 1 — масштабирование коэффициентом y/x даёт поток величины y

Что говорим.

Идея в том, что любой поток раскладывается на путевые компоненты — потоки по путям из источника в сток — и циклические компоненты. При этом величина потока определяется только путевыми компонентами: циклы замкнуты и в величину не вносят вклад. Значит, чтобы получить поток меньшей величины y, достаточно пропорционально уменьшить путевую часть. Разложим f по теореме разложения: он равен сумме α i на P i плюс циклическим компонентам, а сумма коэффициентов α i равна x. Положим f штрих равным y, делённому на x, и умноженному на путевую часть. Это снова поток — линейная комбинация потоков есть поток. Его величина равна y/x, умноженному на сумму α i, то есть равна y. Наконец, y/x меньше единицы, поэтому новые значения на рёбрах строго меньше исходных и точно не выходят за пропускные способности. Поток величины y построен.

Задача 2. Разложение потока на пути и циклы

Условие. *всякий поток f в сети можно представить как сумму потоков по путям s → t и потоков по циклам.*

Что пишем на доске и рисуем.

Доказательство конструктивное: по одному выделяем из потока либо путь s → t, либо цикл, и вычитаем его. Каждое такое вычитание обнуляет хотя бы одно ребро, поэтому за конечное число шагов разложим весь поток.

Ведём индукцию по числу рёбер с f(e) > 0. База: f ≡ 0 — сумма пуста. Шаг: возьмём любое ребро (u, v) с f(u, v) > 0. Запускаем поиск по рёбрам с положительным потоком от v вперёд и от u назад. Сохранение потока гарантирует, что из каждой внутренней вершины есть куда продолжить обход. Возникает ровно один из двух случаев:

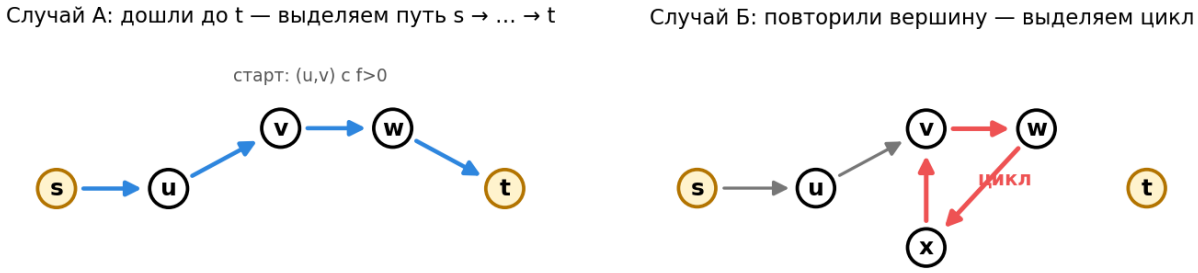

Рисунок 2 — два случая обхода: путь до t либо возврат в посещённую вершину

А. поиск дошёл до t (параллельно из u дотянулся до s) — выделена структура-путь P: s → … → u → v → … → t;

Б. поиск вернулся в уже посещённую вершину — выделен цикл C.

Возьмём узкое место δ = min f(e) по рёбрам структуры и определим новый поток:

$$f_1(e) = f(e) - \delta \text{ на рёбрах структуры;} \quad f_1(e) = f(e) \text{ иначе.}$$

Сохранение потока в f_1 выполняется автоматически (из всех рёбер структуры вычли одинаковый δ). Хотя бы на одном ребре $f_1(e) = 0$ — структура «сжала» его до нуля. По индукции f_1 разлагается в требуемую сумму; добавив к ней исходную структуру с коэффициентом δ, получаем разложение для f. Процесс завершается не более чем за |E| шагов. ■

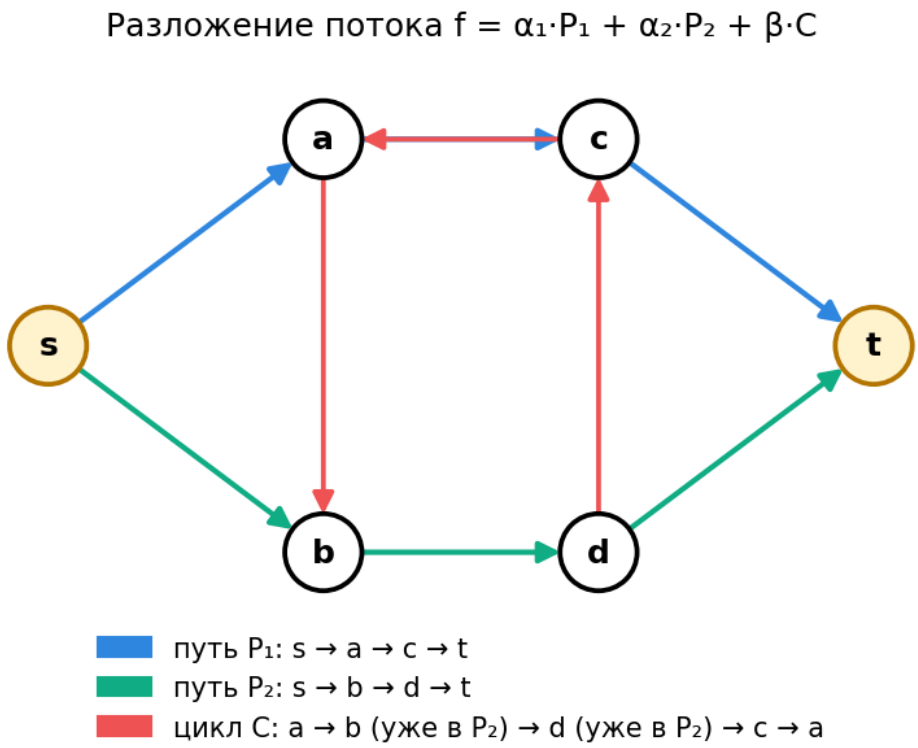

Рисунок 3 — пример: поток раскладывается на два пути и один цикл

Что говорим.

Доказательство конструктивное — покажем, как алгоритмически извлечь разложение. Ведём индукцию по числу рёбер с положительным потоком. База: если поток тождественно нулевой, разложение пустое. Индукционный шаг: берём любое ребро u v с положительным потоком и запускаем обход — от вершины v движемся вперёд по рёбрам с положительным потоком, параллельно от u — назад. Сохранение

потока гарантирует, что из каждой внутренней вершины есть куда продолжить. Возможны два случая. Случай А: поиск дошёл до стока t, а обратный обход от u — до источника s; тогда выделена структура-путь от s до t, проходящая через ребро u v. Случай Б: поиск вернулся в уже посещённую вершину — тогда выделен цикл. В обоих случаях берём минимум потока по рёбрам этой структуры — обозначим δ — и вычитаем его. Условие сохранения не нарушается, потому что из всех рёбер структуры вычли одинаковое число. Хотя бы на одном ребре — там, где был минимум — новый поток обнуляется, число ненулевых рёбер строго уменьшается. По индукции остаток разлагается в сумму; добавив к ней исходную структуру с коэффициентом δ, получаем разложение для f. Процесс завершается не более чем за |E| шагов.

Задача 3. Целочисленный максимальный поток

Условие. *если все пропускные способности c(e) целочисленные, то существует максимальный поток, значения которого на всех рёбрах целые.*

Что пишем на доске и рисуем.

Достаточно показать, что алгоритм Форда–Фалкерсона, начатый с нулевого потока, сохраняет целочисленность на каждой итерации. Из этого автоматически следует, что его результат — целочисленный поток; а по теореме max-flow/min-cut он ещё и максимальный.

Ведём индукцию по номеру итерации.

1. $f_0 \equiv 0$ — целый.

2. пусть f_k целый. Остаточные ёмкости

$$c_res(u, v) = c(u, v) - f_k(u, v) + f_k(v, u)$$

— это разности целых чисел, следовательно, тоже целые.

3. bottleneck увеличивающего пути $\delta = \min c_res(e)$ — минимум целых, значит, целое.

4. обновление $f_{k+1} = f_k \pm \delta$ сохраняет целочисленность.

Завершение. Величина |f| возрастает не меньше чем на 1 за шаг, а сверху ограничена $\Sigma\, c(s, \cdot)$. В момент отсутствия увеличивающих путей поток максимален (теорема Форда–Фалкерсона) и при этом целый. ■

Целочисленность сохраняется на каждой итерации (все δ целые)

Итер.	Событие в остаточной сети	bottleneck δ	|f|
0	стартовый поток $f_0 \equiv 0 \in \mathbb{Z}$	—	0
1	путь s → a → t, c_res = (4, 3)	δ = min(4, 3) = 3	3
2	путь s → b → t, c_res = (5, 2)	δ = min(5, 2) = 2	5
3	путь s → a → b → t, c_res = (1, 3, 2)	δ = 1	6
4	увеличивающих путей нет	—	6 (max)

Рисунок 4 — пример: на каждой итерации bottleneck δ остаётся целым

Что говорим.

Утверждение доказывается через инвариант алгоритма Форда–Фалкерсона: если стартовать с нулевого потока, целочисленность сохраняется на каждой итерации. Ведём индукцию. База: $f_0 \equiv 0$ — целый. Индукционный шаг: пусть поток f_k целый. Остаточные пропускные способности равны c минус f_k u v плюс f_k v u — это разности целых, значит, тоже целые. Bottleneck увеличивающего пути δ — минимум по целым, опять целое. Новый поток $f_{k+1} = f_k \pm \delta$ получается прибавлением или вычитанием целого, значит тоже целый. Остаётся показать завершение: на каждой итерации величина потока возрастает хотя бы на единицу, потому что δ — положительное целое, а сверху она ограничена суммой пропускных способностей исходящих из источника рёбер. Значит, алгоритм завершится за конечное число шагов, и по теореме Форда–Фалкерсона в этот момент поток максимален. По построению он целочисленный.

Задача 4. Контрпример Форда–Фалкерсона на вещественных

Условие. *построить сеть с вещественными пропускными способностями, на которой алгоритм Форда–Фалкерсона при неудачном выборе увеличивающих путей никогда не завершается.*

Что пишем на доске и рисуем.

Форд–Фалкерсон в исходной формулировке не специфицирует, как выбирать увеличивающий путь. Если ёмкости иррациональны, плохой выбор приводит к геометрической прогрессии приростов, сумма которой конечна, но меньше максимального потока — алгоритм не достигает оптимума.

Рассмотрим обратное золотое сечение

$$\varphi = (\sqrt{5} - 1) / 2 \approx 0.618, \quad \text{со свойством } \varphi^2 = 1 - \varphi.$$

В сети s, t и две внутренние вершины a, b. Четыре «толстых» ребра с ёмкостями M соединяют s с a, b и a, b с t. Между a и b — два тонких ребра с ёмкостями 1 и φ.

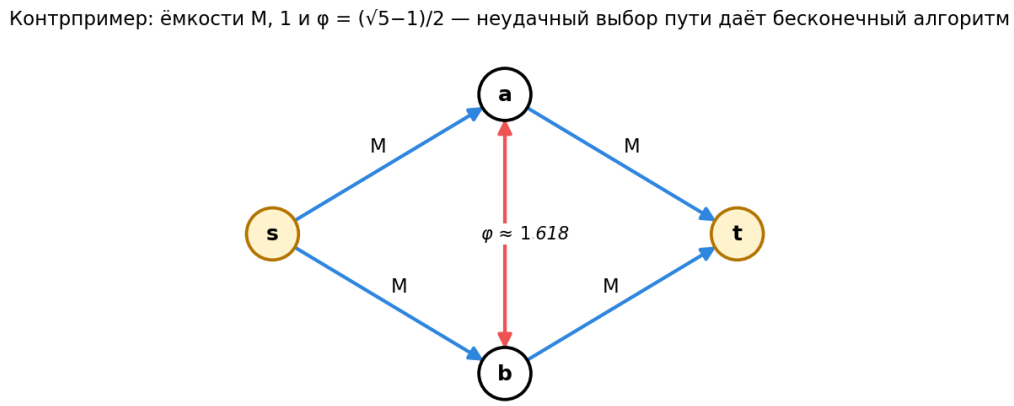

Рисунок 5 — сеть Форда–Фалкерсона: ёмкости M, 1 и φ

При жадном выборе пути (через тонкое ребро с наименьшей оставшейся ёмкостью) приросты потока образуют последовательность

$$1, \varphi, \varphi^2, \varphi^3, \ldots$$

Сумма этого ряда конечна и строго меньше максимального потока 2M + 1, поэтому алгоритм бесконечно итерируется и не сходится к оптимуму.

Что говорим.

Это известный контрпример к наивной версии алгоритма. Форд–Фалкерсон не специфицирует, как выбирать увеличивающий путь — если ёмкости рациональны, алгоритм всегда завершается, а для иррациональных ёмкостей плохой выбор может привести к проблемам. Возьмём обратное золотое сечение φ — корень из пяти минус единица, делённое на два, приблизительно 0.618; оно удовлетворяет уравнению $\varphi^2 = 1 - \varphi$, и это ключевое свойство. Строим сеть: источник s, сток t, две внутренние вершины a и b. Четыре «толстых» ребра ёмкости M — от s к a и b и от a и b к t. Между a и b — два «тонких» ребра ёмкостей 1 и φ. Если на каждой итерации выбирать увеличивающий путь жадно, через тонкое ребро с наименьшей оставшейся ёмкостью, то приросты потока образуют геометрическую прогрессию 1, φ, φ^2, φ^3 и так далее. Сумма такой прогрессии конечна, равна $1 + \varphi/(1 - \varphi)$, что строго меньше максимального потока 2M + 1. Алгоритм бесконечно итерируется, а значение потока сходится к числу, меньшему оптимума. На практике эта проблема снимается алгоритмом Эдмондса–Карпа — он выбирает

увеличивающий путь поиском в ширину, то есть кратчайший, и гарантированно завершается за $O(V \cdot E^2)$ независимо от ёмкостей.

Задача 5. Форд–Фалкерсон в неориентированном графе

Условие. *предложить модификацию алгоритма Форда–Фалкерсона для неориентированного графа.*

Что пишем на доске и рисуем.

Менять сам алгоритм не нужно — меняется только граф. Неориентированное ребро «пропускает» поток в любую сторону, поэтому естественно заменить его на пару встречных направленных той же ёмкости. В оптимальном потоке встречные компоненты всегда можно сократить, значит результат ориентированной задачи корректно интерпретируется как поток в исходном графе.

Каждое неориентированное ребро {u, v} с пропускной способностью c заменяется на два направленных:

(u, v) с ёмкостью c и (v, u) с ёмкостью c.

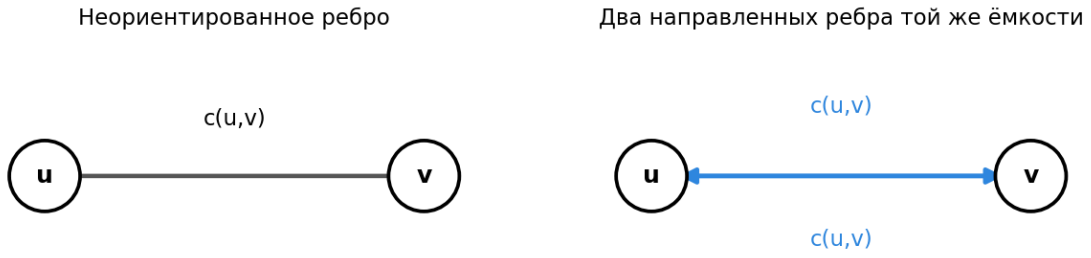

Рисунок 6 — неориентированное ребро заменяется на пару направленных

Корректность. Если в оптимальном потоке $f(u, v) > 0$ и $f(v, u) > 0$ одновременно, вычтем из обоих значений $\min(f(u, v), f(v, u))$ — получится поток той же величины с нулём в одном из направлений. Значит, в оптимуме поток по ребру однонаправленный, и итоговое значение корректно определяется как модуль разности.

Что говорим.

Идея в том, что сам алгоритм Форда–Фалкерсона менять не нужно — меняется только вход. Неориентированное ребро u v с ёмкостью c пропускает поток в любом направлении, поэтому естественно заменить его на пару направленных: (u, v) с ёмкостью c и (v, u) с той же ёмкостью c. Получаем ориентированный граф и применяем обычный

Форд–Фалкерсон. Нужно только проверить корректность — результат ориентированной задачи должен соответствовать потоку в неориентированной. Предположим, в оптимальном потоке одновременно f(u, v) > 0 и f(v, u) > 0. Вычтем из обоих значений min(f(u, v), f(v, u)) — получится допустимый поток той же величины, но теперь одно из направлений обнулено. Значит, в оптимуме поток по ребру однонаправленный, и итоговое значение корректно определяется как модуль разности.

Задача 6. Максимум рёберно непересекающихся путей

Условие. *дан неориентированный граф G и две вершины s и t. Найти максимальное число рёберно непересекающихся путей из s в t.*

Что пишем на доске и рисуем.

Задача сводится к поиску максимального потока. Если на каждом ребре поставить единичную ёмкость, всякий путь пропускает ровно единицу потока, а разложение целочисленного max flow даёт набор единичных путей — это и есть искомые рёберно непересекающиеся пути.

1. каждое ребро {u, v} графа G заменяем на пару направленных (u, v) и (v, u) ёмкости 1;

2. ищем максимальный поток от s к t;

3. раскладываем поток на единичные пути — их число равно |f*| = k.

Корректность — рёберная теорема Менгера:

max{рёберно непересекающихся s-t путей} = min{s-t рёберный разрез}.

Три рёберно непересекающихся пути s → t (каждое ребро имеет ёмкость 1 ⟹ max flow = 3)

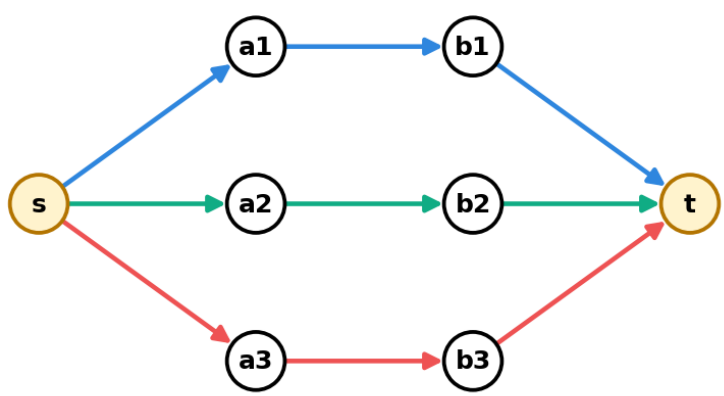

Рисунок 7 — три рёберно непересекающихся пути s → t, каждый несёт единицу потока

Что говорим.

Задача сводится к поиску максимального потока с единичными ёмкостями. Интуиция такая: если каждое ребро пропускает ровно единицу, то всякий поток раскладывается на единичные пути. По теореме о целочисленности максимальный поток целый, и его разложение содержит ровно $|f^*|$ единичных путей из s в t без общих рёбер — это и есть искомые рёберно непересекающиеся пути. Шаги решения: каждое неориентированное ребро заменяем на пару направленных с ёмкостью 1, как в задаче 5; ищем максимальный поток от s к t алгоритмом Форда–Фалкерсона или Эдмондса–Карпа; раскладываем поток на единичные пути процедурой из задачи 2. Количество путей k равно величине максимального потока. Корректность — это рёберная теорема Менгера: максимум рёберно непересекающихся s-t путей равен минимальному размеру s-t рёберного разреза.

Задача 7. Максимум вершинно непересекающихся путей

Условие. *дан неориентированный граф G и две вершины s и t. Найти максимальное число вершинно непересекающихся путей из s в t.*

Что пишем на доске и рисуем.

В графе как таковом нет «ёмкости вершины» — ограничивать можно только рёбра. Приём vertex splitting возвращает задачу в рамки обычного max flow: каждая вершина превращается в два узла, соединённых ребром ёмкости 1, и это ребро становится «бутылочным горлышком» для всех путей через исходную вершину.

Для каждой внутренней вершины $v \notin \{s, t\}$:

— заменяем v на пару v_{in} и v_{out};

— все входящие в v рёбра направляем в v_{in} с ёмкостью ∞;

— все исходящие из v рёбра направляем из v_{out} с ёмкостью ∞;

— внутреннее ребро (v_{in}, v_{out}) имеет ёмкость 1.

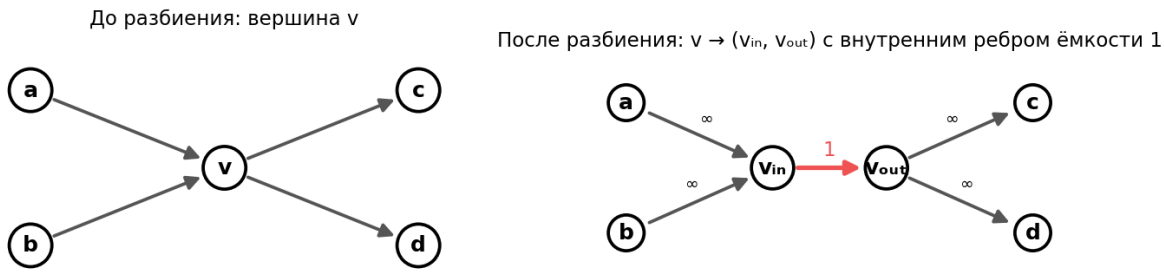

Рисунок 8 — vertex splitting: каждая внутренняя вершина превращается в пару с внутренним ребром ёмкости 1

Далее — максимальный поток от s к t. Вершинная теорема Менгера:

max{вершинно непересекающихся s-t путей} = min{s-t вершинный разрез}.

Что говорим.

В обычной задаче о потоке ограничения ставятся на рёбра — у вершины нет своей ёмкости. Чтобы ограничить прохождение потока через вершину, применяется приём vertex splitting. Для каждой внутренней вершины v — то есть не s и не t — делаем следующее: разбиваем её на две вершины v_{in} и v_{out}; все рёбра, входившие в v, теперь ведут в v_{in} с ёмкостью ∞; все рёбра, исходившие из v, теперь выходят из v_{out}, тоже с ∞; а между v_{in} и v_{out} проводим внутреннее ребро ёмкости 1 — оно становится «бутылочным горлышком» для любого пути через исходную вершину. Два пути через одну и ту же вершину v делили бы это внутреннее ребро — невозможно при ёмкости 1. Поэтому максимум вершинно непересекающихся путей равен величине max flow в полученной сети, и, по вершинной теореме Менгера, это же равно минимальному размеру вершинного s-t разреза.

Задача 8. Задача о замках (минимальный вершинный разрез)

Условие. *на поле n × m клетки бывают свободными, занятыми горами или замками. Построить стены в минимальном числе свободных клеток (не замковых) так, чтобы из одного замка нельзя было пройти в другой (движения по четырём сторонам; горы и стены непроходимы).*

Что пишем на доске и рисуем.

Застройка клетки — это удаление вершины из графа проходимости. Задача — удалить минимум вершин так, чтобы замки оказались в разных компонентах. Это минимальный вершинный s-t-разрез, и по теореме Менгера он равен максимуму

вершинно непересекающихся путей. Последний считается через vertex splitting и max flow — как в задаче 7.

1. каждая свободная клетка — вершина графа; горы в граф не включаем;

2. клетки замков — s и t, их не раздваиваем (их «застраивать» нельзя);

3. к каждой свободной не-замковой клетке применяем vertex splitting с внутренним ребром ёмкости 1 — это и есть «возможность застроить»;

4. соседние клетки (по четырём сторонам) соединяем рёбрами ёмкости ∞;

5. ищем максимальный поток от s к t. Его величина равна минимальному числу клеток, которые нужно застроить.

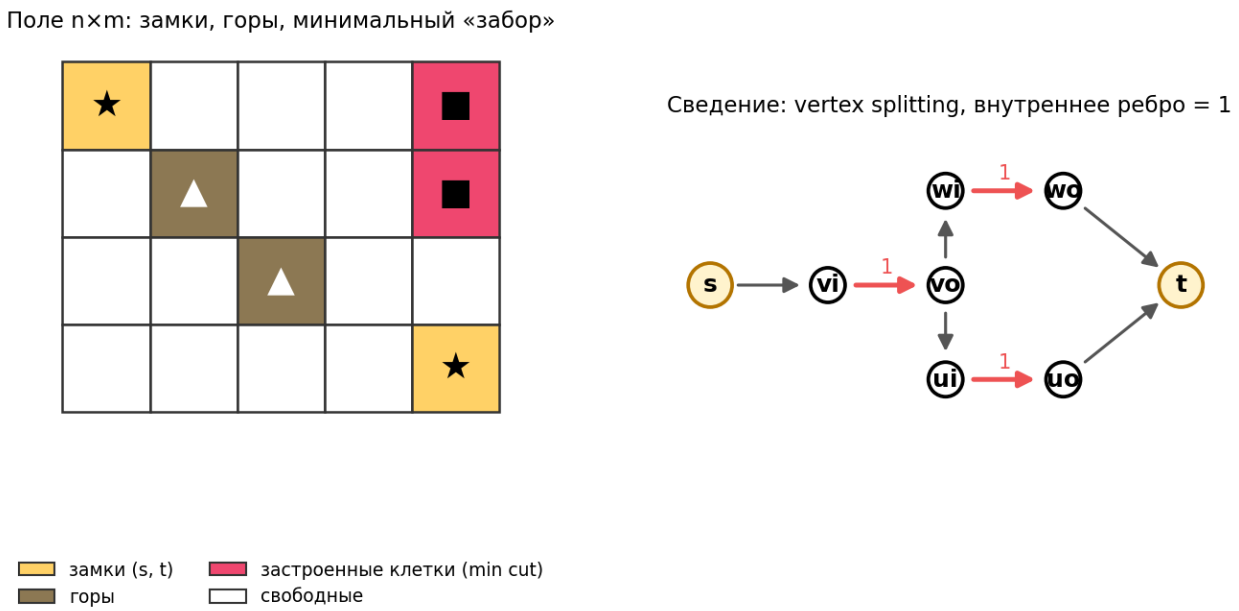

Рисунок 9 — слева пример поля (замки на диагонали, между ними горы ▲), справа граф после vertex splitting

Что говорим.

Задача формулируется в терминах клеток и стен, но сводится к минимальному вершинному разрезу. Застроить клетку — значит удалить соответствующую вершину из графа проходимости. Мы хотим удалить минимум вершин так, чтобы первый замок оказался в одной компоненте связности, а второй — в другой. По теореме Менгера минимальный вершинный s-t разрез равен максимуму вершинно непересекающихся путей от s к t, а он вычисляется через vertex splitting и max flow, как в задаче 7. Построение графа: каждая свободная клетка — вершина, горы в граф не включаем, клетки замков — s и t, и их не раздваиваем, потому что замки застраивать нельзя. К

каждой свободной не-замковой клетке применяем vertex splitting с внутренним ребром ёмкости 1 — это и есть возможность «застроить клетку». Соседние клетки по четырём сторонам соединяем рёбрами ёмкости ∞. Запускаем поиск максимального потока. Его величина и есть ответ — минимальное число клеток, которые нужно застроить.

Задача 9. Фабрики и магазины

Условие. *в ориентированном графе G отмечены k фабрик $F_1, \ldots, F_k$ и k магазинов $S_1, \ldots, S_k$. Нужно соединить каждую фабрику с каким-нибудь магазином (биекция) так, чтобы получившиеся пути не пересекались.*

Что пишем на доске и рисуем.

У задачи несколько источников и стоков. Стандартный приём — добавить суперисточник и суперсток с единичными «лепестками» к фабрикам и магазинам: так каждая фабрика и каждый магазин смогут быть использованы не более одного раза. Непересекаемость путей обеспечивается тем же vertex splitting (если требуются вершинно непересекающиеся пути) или единичными ёмкостями рёбер (если рёберно).

1. добавляем суперисточник S* и проводим рёбра $S^* \to F_i$ с ёмкостью 1;

2. добавляем суперсток T* и проводим рёбра $S_j \to T^*$ с ёмкостью 1;

3. для вершинно непересекающихся путей — vertex splitting на внутренних вершинах с ребром ёмкости 1; для рёберно непересекающихся — всем рёбрам исходного графа ставим ёмкость 1;

4. запускаем алгоритм поиска максимального потока от S* к T*.

Если $|f^*| = k$, разложение потока на единичные пути задаёт искомое соответствие фабрика → магазин. Если $|f^*| < k$ — задача неразрешима.

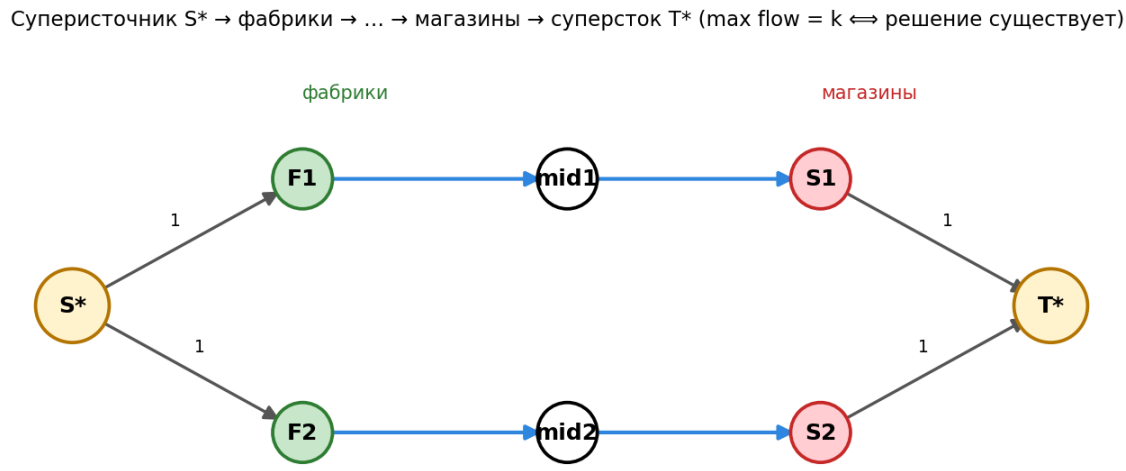

Рисунок 10 — суперисточник S, фабрики, магазины, суперсток T*; max flow = k ⟺ биекция существует*

Что говорим.

У задачи несколько источников и несколько стоков, и стандартный приём — свести её к обычной задаче с одним источником и одним стоком. Добавляем суперисточник S* и проводим рёбра от него к каждой фабрике F_i с ёмкостью 1. Добавляем суперсток T* и рёбра от каждого магазина S_j к T* с той же ёмкостью 1. Единичные ёмкости этих «лепестков» гарантируют, что каждая фабрика и каждый магазин используются не более одного раза — то есть получится биекция. Дальше, в зависимости от того, какие пути нужны: если вершинно непересекающиеся — применяем vertex splitting ко всем внутренним вершинам с ребром ёмкости 1; если рёберно непересекающиеся — даём единичные ёмкости всем рёбрам исходного графа. Запускаем поиск максимального потока от S* к T*. Если величина максимального потока равна k — задача разрешима, а разложение потока на единичные пути даёт явное соответствие фабрика → магазин. Если |f*| меньше k — решения не существует: больше, чем величина максимального потока, пар соединить нельзя.